Московский Физико–Технический Институт

Численные методы
Задание №4

ФИО: Лазарь Владислав Игоревич
Группа: Б01-207а
Почта: lazar.v@phystech.edu

Долгопрудный, 2026 г.

Согласно стандарту `IEEE 754`, число представляется как:

$$N = (-1)^s \cdot (1 + f) \cdot 2^{e - \text{bias}},$$

где:

- Знак s (занимает 1 бит).
- Мантисса $(1 + f)$ определяет точность числа. Причём единица перед дробной частью f не хранится в памяти (скрытый бит), что позволяет сэкономить один бит.
- Экспонента e определяет диапазон числа (занимает, например, 8 бит для `binary32` или 11 бит для `binary64`).
- bias — это фиксированное число, которое прибавляется к реальному значению экспоненты перед его сохранением в памяти компьютера.

При работе с числовыми множествами ограниченной мощности могут появиться очень интересные проблемы. Например:

1. **Loss of Precision.** Когда мы выполняем операцию над двумя числами с разными порядками, процессор должен привести их к общему порядку. Для этого мантисса меньшего числа сдвигается вправо.

   Здесь мы можем пронаблюдать проблему отсутствия ассоциативности по сложению. Попробуем вычислить выражения $[(-2^{15} + 2^{15}) + 2^{-10}]$ и $[-2^{15} + (2^{15} + 2^{-10})]$:

   $$(-2^{15} + 2^{15}) + 2^{-10} = 0 + 2^{-10} = 2^{-10}.$$

   $$-2^{15} + (2^{15} + 2^{-10}) = -2^{15} + (2^{15} + 2^{15} \cdot 2^{-25}) = -2^{15} + 2^{15} = 0.$$

2. **Catastrophic Cancellation.** Рассмотрим два близких числа, каждое из которых уже содержит небольшую погрешность (работаем с точностью до 7 значащих цифр) — $x = 1.234567 \pm 0.0000001$ и $y = 1.234560 \pm 0.0000001$.

   При их вычитании получим:

   $$(x - y) = 0.000007 = 7.000000 \cdot 10^{-6}.$$

   То есть у нас осталась всего одна надёжная цифра. И если исходные x и y были результатами предыдущих вычислений и имели крошечную ошибку в 7-м знаке, эта ошибка теперь стала главной частью результата. Погрешность в данном случае выросла с 0.00001% до почти 100%.

В связи с описанными проблемами, нам важно выбрать устраивающий нас алгоритм для работы с числами с плавающей точкой.

1. **"Быстрый" алгоритм.** Основан на преобразовании формулы дисперсии:

$$D[X] = E[X^2] - \big(E[X]\big)^2 = \frac{1}{n}\left(\sum_{i=1}^{n} x_i^2 - \frac{1}{n}\left(\sum_{i=1}^{n} x_i\right)^2\right).$$

   Это самый нестабильный метод. Если среднее значение μ велико, а стандартное отклонение σ мало, то величины $\sum\limits_{i=1}^{n} x_i^2$ и $\frac{1}{n}\left(\sum\limits_{i=1}^{n} x_i\right)^2$ становятся огромными и очень близкими друг к другу. При их вычитании происходит *Catastrophic Cancellation.*

2. **Двухпроходной алгоритм.** Использует определение дисперсии "в лоб":

$$D[X] = \frac{1}{n}\sum_{i=1}^{n}(x_i - \overline{x})^2.$$

   На первом проходе вычисляется среднее $\overline{x}$. На втором проходе вычисляется сумма квадратов отклонений. Минусом является разве что двойное прочтение данных.

3. **Однопроходной алгоритм.** Это рекуррентный метод, который обновляет оценку дисперсии по мере поступления новых данных.

   Для каждого нового значения x_n:

   (a) Вычисляется новое среднее:

$$\overline{x_n} = \overline{x_{n-1}} + \frac{x_n - \overline{x_{n-1}}}{n}.$$

   (b) Обновляется сумма квадратов отклонений M:

$$M_n = M_{n-1} + \frac{x_n - \overline{x_{n-1}}}{x_n - \overline{x_n}}.$$

   (c) Тогда для дисперсии имеем:

$$D[X] = \frac{M_n}{n}.$$

Влияние потери точности будет видно на предложенных выборках:

1. Имеем $\mu = 1$, $\sigma = 1$. Отношение среднего к отклонению 1:1. Ошибки в `float32` будут минимальны, так как порядки чисел сопоставимы.

2. Имеем $\mu = 10$, $\sigma = 0.1$. Отношение среднего к отклонению 100:1. Квадраты чисел будут в 10^4 раз больше дисперсии. В `float32` (7 значащих цифр) начнется потеря точности в "быстром" методе.

3. Имеем $\mu = 100$, $\sigma = 0.01$. Отношение среднего к отклонению 10000:1. Квадрат среднего 10^4 соотносится с дисперсией 10^{-4} как 10^8. Так как `float32` хранит только ~ 7 значащих цифр, "быстрый" алгоритм в этом случае может выдать нулевую или даже отрицательную дисперсию, так как информация о разности в 8-м знаке будет просто утеряна.

Результаты:

С кодом можно ознакомиться в `./FP.py`. Относительная ошибка вычислялась относительно эталонного значения, полученного *однопроходным алгоритмом* в формате `float64` для каждой конкретной выборки. Так я изолировал вычислительную погрешность формата `float32` от статистической ошибки генерации выборки.

Выборка (μ, σ)	**Тип**	**Метод**	**Отн. ошибка (%)**
(1, 1)	`float32`	Быстрый	0.0000095448
		Двухпроходной	0.0000041304
		Однопроходной	0.0000002964
	`float64`	Быстрый	0.0000000000
		Двухпроходной	0.0000000000
		Однопроходной	0.0000000000
(10, 0.1)	`float32`	Быстрый	0.0486862700
		Двухпроходной	0.0000216527
		Однопроходной	0.0000192231
	`float64`	Быстрый	0.0000000001
		Двухпроходной	0.0000000000
		Однопроходной	0.0000000000
(100, 0.01)	`float32`	Быстрый	165.2863777129
		Двухпроходной	0.0003588469
		Однопроходной	0.0003786767
	`float64`	Быстрый	0.0000028299
		Двухпроходной	0.0000000000
		Однопроходной	0.0000000000